CS344 Operating Systems Lab 2 Report Drive link:https://drive.google.com/file/d/18NtysYkbtRuvBNUbSpAJMMhAo5dvStU_/view?usp=sharing

GROUP-18:

Vislavath Bindhu (230101112)
Tippana Likitha Reddy (230101105)
Bommagani Rohitha (230101027)
Annadevara Dedeepya Siri (230101011

Task 2.1: Weighted Round Robin (WRR) Scheduler

Objective

The goal of this task is to enhance the standard round-robin scheduler in the xv6 operating system. The new scheduler will implement a **Weighted Round Robin (WRR)** algorithm, which allocates CPU time to processes based on user-defined priorities. This ensures that higher-priority processes get more CPU time while preventing lower-priority processes from being starved. A default priority will be assigned to all new processes.

Implementation Details

To implement the WRR scheduler, new system calls were added, the process control block (struct proc) was updated, and the core scheduling logic was modified.

System Call Implementation

Two new system calls were introduced to allow user-space programs to manage process priorities:

- int set_priority(int n): Sets the priority (weight) of the calling process to n.
- int get_priority(void): Returns the current priority of the calling process.

The implementation process followed these standard xv6 steps:

1. **user/user.h**: System call prototypes were added.
2. **user/usys.pl**: This script was used to generate the assembly stubs in user/usys.S.

```
39  enter("set_priority");
40  enter("get_priority");
41  entry("getrticks");
42
43
```

3. **kernel/syscall.h & kernel/syscall.c**: The system call numbers and function mappings were added to the kernel.

4. **kernel/sysproc.c**: The kernel-side logic for the system calls was implemented here.

```
  //added.....Lab2.....
  //added.....Lab2.....

uint64
sys_set_priority(void)
{
  int w;
  argint(0, &w);       // get first argument
  struct proc *p = myproc();
  p->weight = w;

  printf("sys_set_priority: pid=%d got arg %d\n", myproc()->pid, w);
  return p->pid;
}

// Get the priority of the current process.
int
sys_get_priority(void)
{
  int priority;
  struct proc *p = myproc();

  acquire(&p->lock);
  priority = p->weight;
  release(&p->lock);

  return priority;
}

uint64
sys_getrticks(void)
{
  struct proc *p = myproc();
  return p->rticks;
}
```

Kernel Modifications

To support WRR, the process structure was updated to store priority information, and the process allocation function was modified to set a default priority.

- **Process Structure (kernel/proc.h)**: A weight field was added to struct proc to store the priority of each process. A ticks_run counter was also added to track the time slices used by a process.

kernel > C proc.h > proc

```
84  // Per-process state
85  struct proc {
86    struct spinlock lock;
87
88    uint ticks; // declaring ticks
89
90    // p->lock must be held when using these:
91    enum procstate state;        // Process state
92    void *chan;                  // If non-zero, sleeping on chan
93    int killed;                  // If non-zero, have been killed
94    int xstate;                  // Exit status to be returned to parent's wait
95    int pid;                     // Process ID
96
97    // wait_lock must be held when using this:
98    struct proc *parent;         // Parent process
99
100   // these are private to the process, so p->lock need not be held.
101   uint64 kstack;               // Virtual address of kernel stack
102   uint64 sz;                   // Size of process memory (bytes)
103   pagetable_t pagetable;       // User page table
104   struct trapframe *trapframe; // data page for trampoline.S
105   struct context context;      // swtch() here to run process
106   struct file *ofile[NOFILE];  // Open files
107   struct inode *cwd;           // Current directory
108   char name[16];               // Process name (debugging)
109
110   //added.....Lab2.....
111   int weight;       // Priority of the process (weight for WRR)
112   int ticks_run;            // Remaining tickets for the time slice
113   int rticks;
114
115 };
116
```

- **Process Initialization (kernel/proc.c)**: The allocproc() function was updated to assign a **default priority of 1** to every new process.

```
126
127   //added.....Lab2.....
128   p->weight = 1;  // default weight
129   p->ticks_run = 0;  // no active slice yet
130
```

Scheduler Logic Modification

The core WRR logic was implemented in the timer interrupt handler in kernel/trap.c. On each timer tick for a running process, a counter (ticks_run) is incremented. When the number of ticks run meets or exceeds the process's weight, the counter is reset, and the process yields the CPU. This ensures processes with a higher weight run for proportionally longer.

```
  //added.....Lab2.....

  // give up the CPU if this is a timer interrupt.
  if (which_dev == 2) {
    if (p && p->state == RUNNING) {
      acquire(&p->lock);
      p->ticks_run++;
      release(&p->lock);
      if (p->ticks_run >= p->weight) {
        acquire(&p->lock);
        p->ticks_run = 0;
        release(&p->lock);
        yield();
      }
    }
  }

  usertrapret();
}
```

```

  //added.....Lab2.....

  struct proc *p;
  for(p = proc; p < &proc[NPROC]; p++){
      acquire(&p->lock);
      if(p->state == RUNNING){
        p->rticks++;
      }
      release(&p->lock);
    }

  w_stimecmp(r_time() + 1000000);
}
```

Testing and Verification

To verify the WRR scheduler, a user program (test.c) was created to demonstrate that processes with higher priorities receive proportionally more CPU time.

- **Test Program (test.c)**: This program forks three child processes and assigns them priorities of 10, 1, and 5. Each process then runs in a loop, printing an identifier to the console.

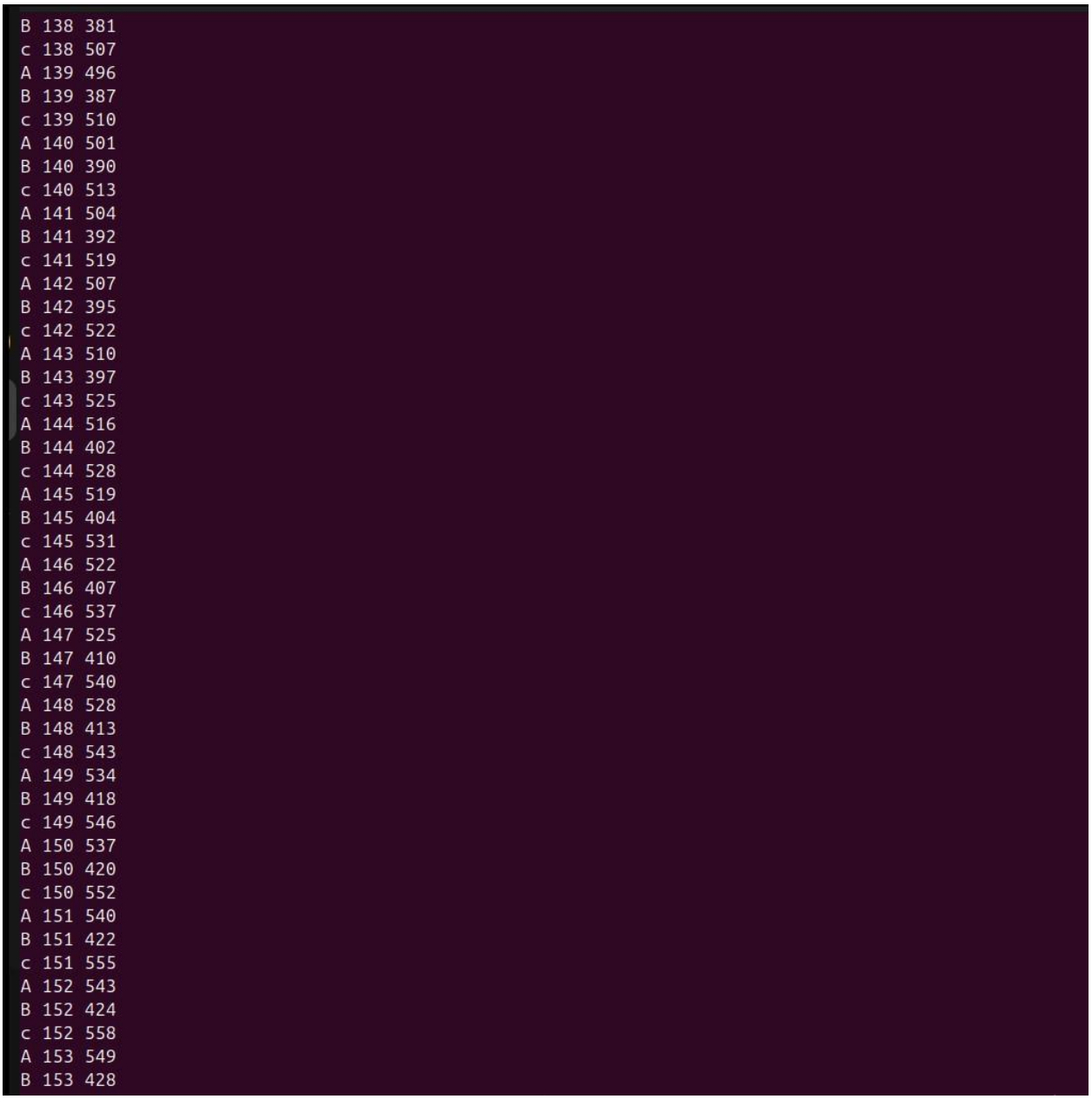
```c
volatile int sink;
void spin(int n){ for(volatile int i=0;i<n;i++) sink += i; }

int main(void) {
  if (fork() == 0) {
    int i = 0;
    set_priority( 10);
    for(;;) { spin(40000000); printf("A %d %d\n", ++i, getrticks()); }
  }
  if (fork() == 0) {
    set_priority( 1);
    int j= 0;
    for(;;) { spin(40000000); printf("B %d %d\n", ++j, getrticks()); }
  }
  if (fork() == 0) {
    set_priority( 5);
    int k= 0;
    for(;;) { spin(40000000); printf("c %d %d\n", ++k, getrticks()); }
  }
  wait(0); wait(0); wait(0);
  exit(0);
}
```

- **Makefile**: The test program was added to the UPROGS section of the Makefile to ensure it is compiled and available in the xv6 user space.

- **Output Analysis**: The output shows the execution of the test program. Process A (priority

10) prints most frequently, Process C (priority 5) prints about half as often, and Process B (priority 1) prints the least. This confirms the correct implementation of the WRR scheduler.



Task 2.2: Implement a Custom Scheduling Algorithm (SRTF)

Algorithm Choice and Justification

For the advanced task, the **Shortest Remaining Time First (SRTF)** scheduling algorithm was implemented.

- **Justification**: SRTF is the preemptive version of Shortest Job First (SJF) and is optimal for minimizing the average waiting time of processes. Its preemptive nature makes it highly responsive, as it can immediately switch to a newly arrived process if it has a shorter burst time than the one currently running.
- **Trade-offs**: The main disadvantage of SRTF is the risk of **starvation**, where long processes may be indefinitely preempted by a continuous stream of shorter ones. Furthermore, it requires an accurate prediction of a process's total CPU burst time. For this implementation, the user process provides this estimate via a system call.

Implementation Details

System Call and Kernel Modifications

- **set_burst(int n) System Call**: A new system call was added to allow a user process to declare its estimated CPU burst time. [Screenshot of your set_burst() implementation in kernel/sysproc.c]
- **Process Structure (kernel/proc.h)**: The struct proc was modified to store the necessary information for SRTF scheduling:
    - burst: The total CPU burst time, provided by the user.
    - runtime: Total CPU time consumed so far.
    - remaining: The estimated time left for the process (burst - runtime). [Screenshot of the modified struct proc for SRTF in kernel/proc.h]

Scheduler and Preemption Logic

- **SRTF Scheduler (kernel/proc.c)**: The scheduler() function was completely replaced. Instead of a round-robin loop, the new scheduler iterates through all RUNNABLE processes to find the one with the **minimum remaining time** and schedules it for execution.

```
//---------------------------------LAB 2--------------- IMPLEMENTING SRTF-----------------------------------
void
scheduler(void)
{
  // pointer to proc
  struct proc *shortest;
  struct proc *p;
  struct cpu *c = mycpu();

  c->proc = 0;
  for(;;){
    // Make sure interrupts are on while we scan/run processes.
    intr_on();

    shortest = 0;
    int best_rem = SRTF_INF;

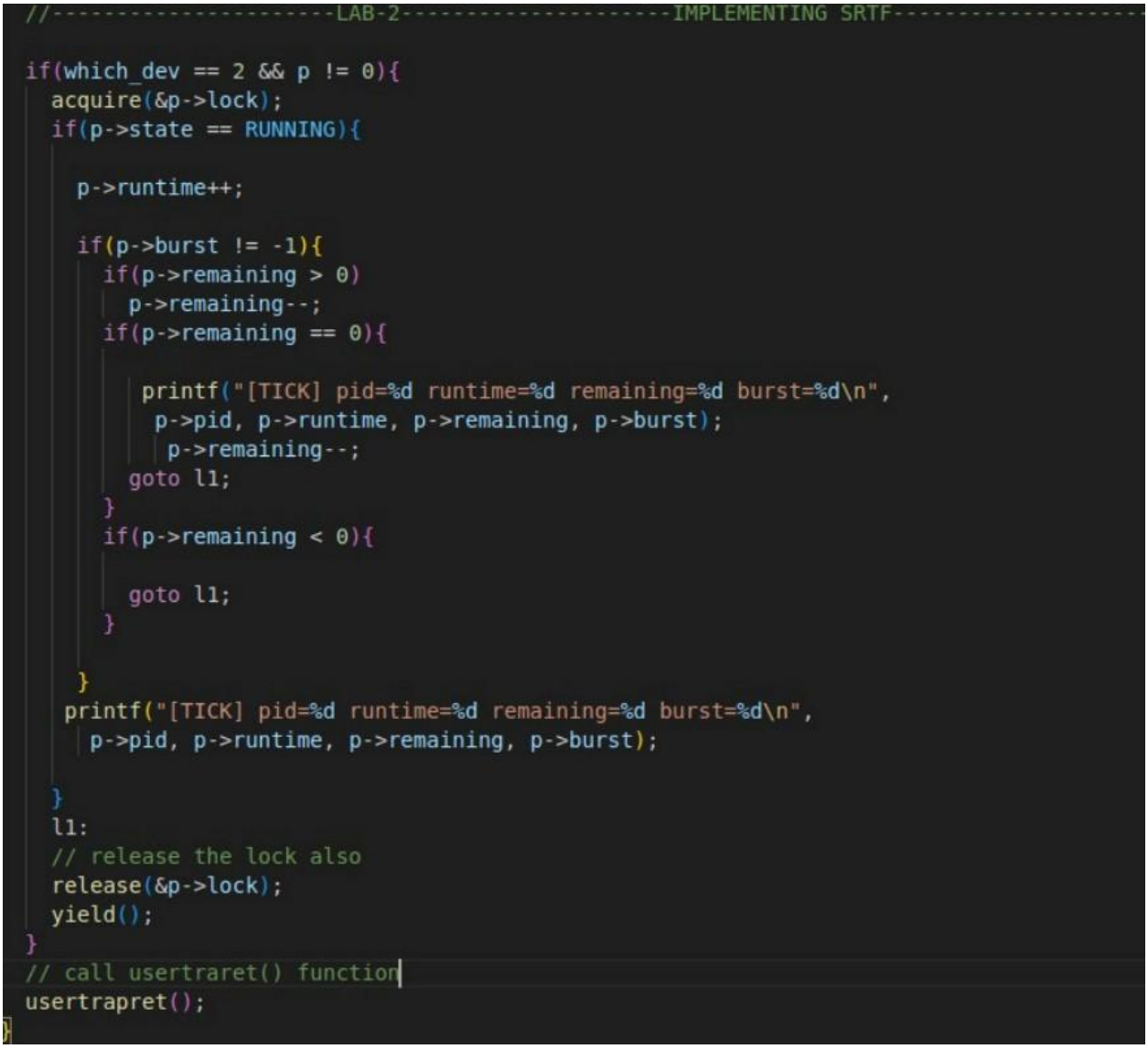
    for(p = proc; p < &proc[NPROC]; p++){
      acquire(&p->lock);
      if(p->state == RUNNABLE){
        int rem = p->remaining;
        /* If burst == -1, treat as very large (unknown length). */
        if(p->burst == -1) rem = SRTF_INF;
        // printf("scheduler: all pid=%d remaining=%d burst=%d\n",
        //     p->pid, p->remaining, p->burst); // <- CORRECTED
        if(shortest == 0 || rem < best_rem ||
           (rem == best_rem && p->pid < shortest->pid)) {
          shortest = p;
          best_rem = rem;
        }
      }
      release(&p->lock);
    }

    if(shortest != 0){
      acquire(&shortest->lock);
      if(shortest->state == RUNNABLE){
        shortest->state = RUNNING;
        c->proc = shortest;
        //     shortest->pid, shortest->remaining, shortest->burst);
        // printf("scheduler: choosing pid=%d remaining=%d burst=%d\n",
        // When the process yields/blocks/exits and we come back:

        // Context switch to the chosen process (holds shortest->lock across switch).
        swtch(&c->context, &shortest->context);

        c->proc = 0;
      }
      release(&shortest->lock);
    } else {
     // this will wait for the interruupt
      asm volatile("wfi");
    }
```

- **Preemption in Timer Interrupt (kernel/trap.c)**: To enable preemption, the timer interrupt handler in kernel/trap.c was modified. On every tick, the current process's runtime is incremented and remaining time is decremented. A call to yield() then forces a context switch, allowing the scheduler to re-evaluate and preempt the current process if a new, shorter job has become available.



```
//----------------------LAB-2--------------------IMPLEMENTING SRTF-------------------

if(which_dev == 2 && p != 0){
  acquire(&p->lock);
  if(p->state == RUNNING){

    p->runtime++;

    if(p->burst != -1){
      if(p->remaining > 0)
        p->remaining--;
      if(p->remaining == 0){

        printf("[TICK] pid=%d runtime=%d remaining=%d burst=%d\n",
         p->pid, p->runtime, p->remaining, p->burst);
          p->remaining--;
        goto l1;
      }
      if(p->remaining < 0){

        goto l1;
      }

    }
   printf("[TICK] pid=%d runtime=%d remaining=%d burst=%d\n",
     p->pid, p->runtime, p->remaining, p->burst);

  }
  l1:
  // release the lock also
  release(&p->lock);
  yield();
}
// call usertraret() function
usertrapret();
}
```

Testing and Verification

An srtf_test.c program was created to validate the scheduler's correctness by creating processes with different arrival and burst times.

- **Test Program (testing_srtf.c)**: The program forks three processes with specific burst times and start delays.
  - Process 1: Burst = 9, arrives at tick 10.
  - Process 2: Burst = 5, arrives at tick 12.
  - Process 3: Burst = 2, arrives at tick 12.

```c
// ---------------------LAB-2----------------------IMPLEMENTING SRTF---------------------
// Simulated CPU-intensive workload
void run_cpu_task(int proc_id, int burst_len) {
    volatile int dummy = 0;
    bool printed = false;

    for (long long step = 0; step < (long long)burst_len * 100000000; step++) {
        if (step % 10000000 == 0 && !printed) {
            printf("Process %d executing with burst time %d\n", proc_id, burst_len);
            printed = true;
        }
        dummy++;  // keeps CPU busy
    }

    printf("Process %d completed execution.\n", proc_id);
    exit(0);
}

int main(void) {
    printf("== Scheduler Experiment: Different Burst Lengths ==\n");

    int child;

    // Process A
    if ((child = fork()) == 0) {
        set_burst(4);
        sleep(10);
        run_cpu_task(getpid(), 4);
    }

    // Process B
    if ((child = fork()) == 0) {
        set_burst(7);
        sleep(12);
        run_cpu_task(getpid(), 7);
    }

    // Process C
    if ((child = fork()) == 0) {
        set_burst(2);
        sleep(8);
        run_cpu_task(getpid(), 2);
    }

    // Wait for all children
    while (wait(0) > 0);

    printf("== Scheduler Test Finished ==\n");
    exit(0);
}
```

- **Output Analysis**: The output demonstrates the SRTF logic: the scheduler starts with the first process but **preempts** it as soon as a shorter job (Process 3, burst 2) becomes available. It runs the shortest jobs to completion before returning to the longer ones. This sequence confirms the correct implementation

```
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -s
virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ testing_srtf
Running scheduler test with different burst times.
sys_set_burst: pid=4 burst=5 runtime=0 remaining=5

sys_set_burst: pid=5 burst=2 runtime=0 remaining=2

sys_set_burst: pid=6 burst=9 runtime=0 remaining=9

Process 6 working... burst: 9
[TICK] pid=6 runtime=1 remaining=8 burst=9
[TICK] pid=6 runtime=2 remaining=7 burst=9
[TICK] pid=6 runtime=3 remaining=6 burst=9
PProrocecessss  54 w woorrkkiinngg......  bbuurrsstt: : 2
5
[TICK] pid=6 runtime=4 remaining=5 burst=9
[TICK] pid=5 runtime=1 remaining=1 burst=2
[TICK] pid=4 runtime=1 remaining=4 burst=5
[TICK] pid=4 runtime=2 remaining=3 burst=5
[TICK] pid=5 runtime=2 remaining=0 burst=2
[TICK] pid=6 runtime=5 remaining=4 burst=9
[TICK] pid=4 runtime=3 remaining=2 burst=5
[TICK] pid=6 runtime=6 remaining=3 burst=9
[TICK] pid=4 runtime=4 remaining=1 burst=5
[TICK] pid=6 runtime=7 remaining=2 burst=9
[TICK] pid=4 runtime=5 remaining=0 burst=5
[TICK] pid=6 runtime=8 remaining=1 burst=9
[TICK] pid=6 runtime=9 remaining=0 burst=9
Process 5 finished.
Process 4 finished.
Process 6 finished.
Scheduler test completed.
$
```